DOCUMENTO TÉCNICO · TRABAJO AGENTE

John Dark Soul

Agente inteligente sobre el lore de Dark Souls I, II y III construido con LangChain, LangGraph y RAG, con un modelo de lenguaje local.

Integrantes

Santiago Cardozo · Santiago Toro

Aplicación

https://johndarksoul.netlify.app

API

https://api.13-58-191-152.sslip.io/docs

Repositorio

https://github.com/Chuluet/john-dark-soul

Fecha

Octubre de 2026

Contenido

1. Nombre del proyecto e integrantes
2. Tema y descripción del problema
3. Arquitectura general
4. Tecnologías utilizadas
5. Backend
6. Frontend
7. Agente y flujo de LangGraph
8. Modelo local y base de datos vectorial
9. Carga de documentos
10. Estrategias contra alucinaciones
11. Seguridad y autenticación
12. Despliegue
13. Evidencias de funcionamiento
14. Problemas encontrados y conclusiones

1. Nombre del proyecto e integrantes

Proyecto	**John Dark Soul**: agente conversacional que responde preguntas sobre la historia, los personajes, los jefes y los lugares de la trilogía Dark Souls usando únicamente su base de conocimiento.
Integrantes	Santiago Cardozo Santiago Toro
Modalidad	Pareja
Repositorio	https://github.com/Chuluet/john-dark-soul
Frontend desplegado	https://johndarksoul.netlify.app
Backend desplegado	https://api.13-58-191-152.sslip.io (Swagger en `/docs`)

2. Tema y descripción del problema

Tema: el lore de Dark Souls (DS1, DS2 y DS3). La saga casi no cuenta su historia de forma directa: está repartida en descripciones de objetos, diálogos breves y detalles del escenario, y la comunidad la reconstruye en wikis, foros y videos. Quien quiere entenderla tiene que cruzar muchas fuentes, y los nombres se repiten entre juegos (Gwyn, Andre, Patches) o tienen varias traducciones.

Problema: un modelo de lenguaje general responde estas preguntas con seguridad aunque se equivoque: mezcla juegos, inventa relaciones entre personajes o completa huecos con detalles plausibles pero falsos. Para un tema donde la mayoría de los datos son poco conocidos, eso hace que sus respuestas no sean confiables.

Objetivo: construir una aplicación web en la que un usuario autenticado pregunte por el lore y reciba respuestas que:

- se basan solo en documentos recuperados de una base vectorial propia (RAG), no en el conocimiento interno del modelo;
- citan el archivo del que sale cada dato;
- declaran explícitamente cuando no hay información suficiente: "*No tengo suficiente información en mi base de conocimiento para responder esa pregunta.*";
- pueden ampliarse: el usuario sube sus propios documentos (texto, PDF, imágenes, audio o video) y el agente los usa de inmediato.

La base de conocimiento inicial tiene **407 archivos**: 401 páginas de la wiki de la comunidad convertidas a Markdown (con título, juego y URL de origen), 5 imágenes con texto sobre el lore y un video de casi tres horas que resume la saga. Una vez procesados son **4196 fragmentos** en la base vectorial.

3. Arquitectura general

La aplicación se divide en tres partes que se comunican por HTTPS: el **frontend** (React, servido como archivos estáticos desde Netlify), el **servidor de autenticación** (Keycloak) y el **backend** (FastAPI con el agente LangGraph), que a su vez usa el modelo local de Ollama y PostgreSQL con pgvector. Todo lo del servidor corre en una instancia EC2 de AWS con Docker Compose, detrás de Caddy, que es el único servicio expuesto a internet.

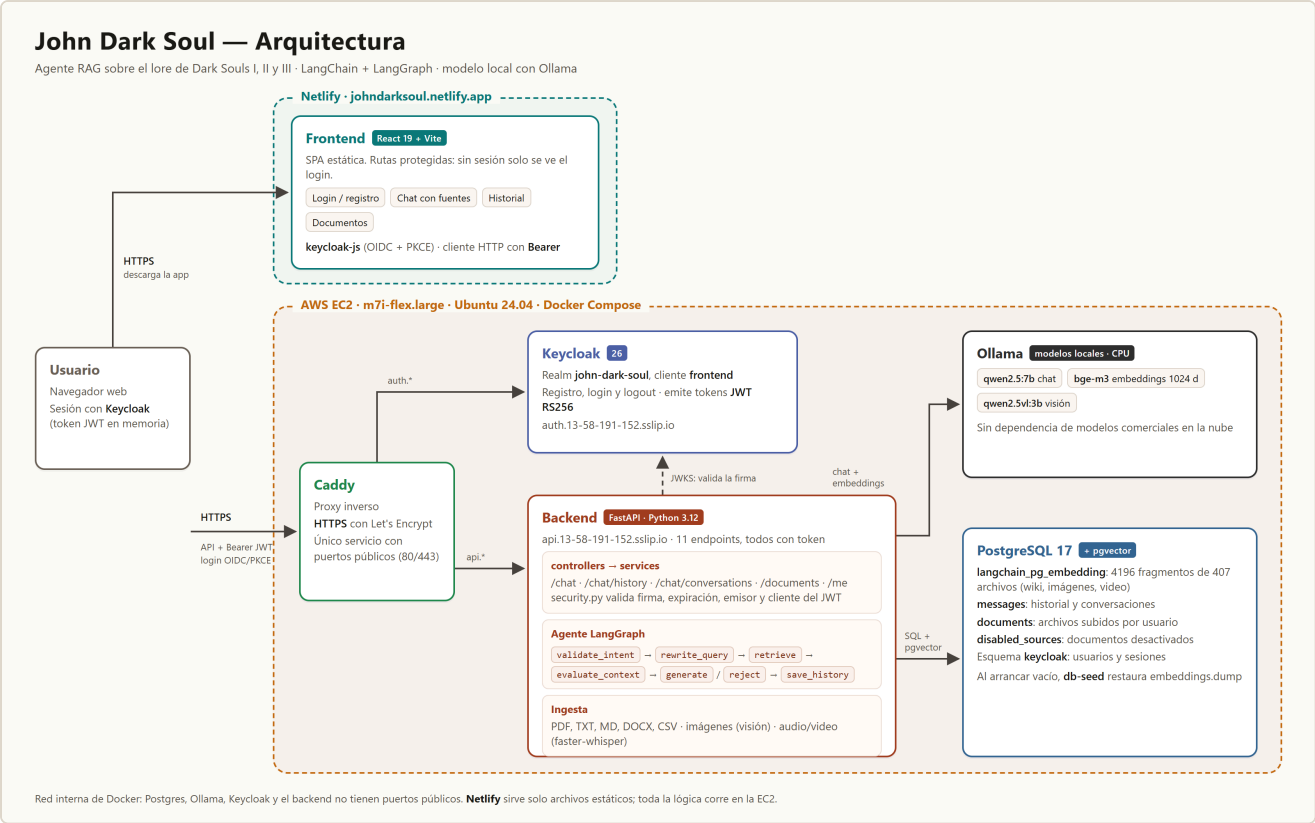


Figura 1. Arquitectura desplegada (docs/architecture.png).

Flujo de una pregunta

1. El usuario inicia sesión: el frontend lo redirige a Keycloak (OIDC con PKCE) y recibe un token JWT.
2. El frontend envía `POST /chat` con la pregunta y el token en `Authorization: Bearer`.
3. El backend valida el token contra las llaves públicas del realm (JWKS) y pasa la pregunta al servicio de chat.
4. El agente LangGraph valida la intención, busca en pgvector, evalúa si el contexto alcanza y genera la respuesta con `qwen2.5:7b`, o la rechaza.
5. La interacción se guarda en la tabla `messages` y la respuesta vuelve con sus fuentes.

Organización del código

El backend sigue una arquitectura por capas, donde cada capa solo habla con la de abajo: `controllers → services → agents / vectorstore / db`.

```
backend/
├── app/          main.py (FastAPI, CORS, routers), config.py (único que lee el entorno),
│                 security.py (validación del JWT), db.py (tablas propias), schemas.py
├── controllers/  endpoints HTTP, sin lógica (auth, chat, documents)
├── services/     lógica de negocio (chat, historial, documentos, ingesta, errores)
├── agents/       grafo LangGraph: state, nodes, graph, prompts, tools, llm
├── vectorstore/  embeddings (bge-m3) y acceso a pgvector
└── tests/        85 pruebas con pytest
frontend/src/     auth (Keycloak), api (cliente HTTP), pages, components
deploy/           Caddyfile, plantilla .env de producción, script de Keycloak
docs/             arquitectura, documento técnico, presentación, capturas, despliegue
```

4. Tecnologías utilizadas

Capa	Tecnología	Uso en el proyecto
Agente	LangChain, LangGraph	Prompts, herramienta de búsqueda (`@tool`), grafo de estados con edges condicionales y memoria por sesión (checkpointer).
Modelos locales	Ollama: `qwen2.5:7b`, `bge-m3`, `qwen2.5vl:3b`; faster-whisper	Chat y razonamiento; embeddings multilingües de 1024 dimensiones; lectura y descripción de imágenes; transcripción de audio y video.
Base vectorial	PostgreSQL 17 + pgvector, `langchain_postgres.PGVector`	Fragmentos con su embedding y metadatos (JSONB); búsqueda por similitud de coseno. Las tablas propias (historial, documentos) viven en la misma base.
Backend	Python 3.12, FastAPI, uv, SQLAlchemy, PyJWT	API REST con 11 endpoints, validación con Pydantic, documentación automática en Swagger.
Ingesta	pypdf, docx2txt, CSVLoader, RecursiveCharacterTextSplitter	Extracción por formato y fragmentación de 1000 caracteres con 150 de solapamiento.
Autenticación	Keycloak 26	Registro, login y logout; tokens JWT RS256; cliente público con PKCE.
Frontend	React 19, Vite 7, React Router 7, keycloak-js 26	Interfaz tipo chatbot, historial, gestión de documentos, rutas protegidas.
Infraestructura	Docker Compose, Caddy, sslip.io	Un compose para local y otro encima para producción; HTTPS automático con Let's Encrypt; dominios sin comprarlos.
Despliegue	Netlify, AWS EC2 (m7i-flex.large)	Frontend estático y servidor con todo el backend.
Calidad	pytest, GitHub	85 pruebas automáticas; trabajo por ramas y pull requests.

5. Backend

El backend es una API REST con FastAPI. **Todos** sus endpoints dependen de `get_current_user`: sin un token válido responden 401. Una prueba automática recorre el esquema OpenAPI y falla si alguna ruta queda accesible sin token.

Método	Ruta	Qué hace
GET	`/me`	Usuario autenticado (`sub`, nombre, correo, roles).
POST	`/chat`	Envía una pregunta al agente; devuelve `answer`, `sources` y `rejected`.
GET	`/chat/history`	Historial de preguntas y respuestas del usuario.
GET	`/chat/conversations`	Conversaciones del usuario agrupadas por `session_id`.

GET	`/chat/conversations/{session_id}`	Mensajes de una conversación, para retomarla.
GET	`/documents`	Archivos que subió el usuario.
POST	`/documents`	Sube un archivo, lo procesa y guarda sus embeddings.
PATCH	`/documents/{id}`	Activa o desactiva un archivo propio.
DELETE	`/documents/{id}`	Borra el archivo y sus fragmentos de la base vectorial.
GET	`/documents/knowledge-base`	Archivos de la base original con su estado para el usuario.
PATCH	`/documents/knowledge-base`	Activa o desactiva un archivo de la base original (solo para ese usuario).

Decisiones de diseño

- **Errores de negocio con su código HTTP.** Los servicios no conocen FastAPI: lanzan errores propios (`services/errores.py`) que declaran su código, y un único manejador los convierte en respuesta: 404 documento ajeno o inexistente, 413 archivo mayor a `MAX_UPLOAD_MB`, 415 formato no soportado, 422 archivo vacío o dañado, 503 Ollama o Postgres caídos. Un rechazo del agente no es un error: responde 200 con `rejected: true`.
- **Datos por usuario.** El `sub` del token identifica al usuario: historial, conversaciones, documentos subidos y documentos desactivados son de cada uno. La base de conocimiento es compartida.
- **Configuración solo por variables de entorno**, leídas en `app/config.py` y documentadas en `.env.example`.
- **Tablas propias** en PostgreSQL: `messages` (historial con `session_id`), `documents` (archivos subidos) y `disabled_sources` (documentos desactivados por usuario). Se crean al primer uso, con un *advisory lock* para que peticiones simultáneas no choquen.

6. Frontend

Aplicación de una sola página en React 19 con Vite, con estética inspirada en el juego (la hoguera, personajes en estilo chibi). Sin sesión solo se muestra la pantalla de login: ninguna ruta de la app es accesible.

Página	Funciones
Login	Botones *Sign in* y *Create account* que llevan a Keycloak. Al volver, la sesión queda activa y el token se renueva solo antes de expirar.
Chat	Interfaz tipo chatbot: preguntas sugeridas, respuesta con párrafos y las fuentes como etiquetas bajo cada respuesta, aviso destacado cuando el agente rechaza la pregunta (con enlace a subir documentos) y mensajes de error si un servicio falla. El menú lateral muestra las conversaciones recientes, que se pueden reabrir para seguir hablando.
History	Preguntas agrupadas por día, buscador, respuesta completa con fuentes al expandir y botón para abrir la conversación en el chat.
Documents	Zona de arrastrar y soltar con barra de progreso, tabla de documentos propios (tipo, juegos detectados, fragmentos, activar/desactivar, borrar) y tabla de la base original con buscador e interruptor por documento.

Integración con la API y Keycloak

- `auth/keycloak.js` inicializa `keycloak-js` con `check-sso` y PKCE (S256), y renueva el token cuando le quedan menos de 30 segundos.
- `api/client.js` agrega `Authorization: Bearer` a cada petición y convierte los errores en mensajes legibles; con un 401 vuelve al login.
- La subida usa `XMLHttpRequest` para mostrar el progreso.
- Las URLs de la API y de Keycloak son variables `VITE_*`, fijadas al compilar. `VITE_USE_MOCKS=true` permite usar la interfaz con datos de ejemplo, sin backend.

7. Agente y flujo de LangGraph

El agente es un `StateGraph` de LangGraph con ocho nodos y edges condicionales. El flujo es explícito en código: el modelo no decide si busca o no, y no hay ningún camino que llegue a `generate` sin pasar por la búsqueda y la evaluación del contexto.

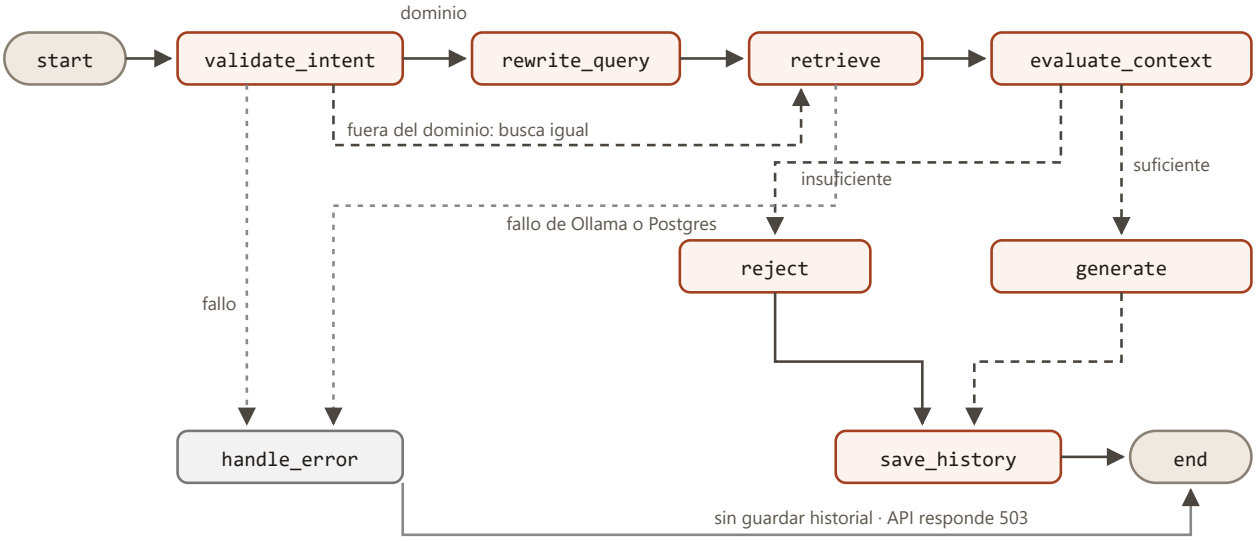


Figura 2. Grafo del agente (`backend/agents/graph.py`). Línea continua: edge fijo; discontinua: edge condicional; gris: manejo de fallos.

Nodo	Qué hace
`validate_intent`	Detecta el idioma (español, inglés u otro) y clasifica con el LLM si la pregunta es del lore. Decide **cómo** se busca, no **si** se busca: con "sí" la pregunta se reescribe; con "no" se busca igual con la pregunta original, porque el clasificador no conoce todos los nombres del juego ni los temas de los documentos que suben los usuarios.
`rewrite_query`	Traduce la pregunta al inglés con los nombres oficiales ("Vigilantes del Abismo" → "Abyss Watchers") y resuelve referencias a la conversación ("¿y su lobo?" → "Who was Artorias's wolf?").
`retrieve`	Busca en pgvector con la pregunta original, la reescrita y la reescrita con nombres completos tomados de los títulos de la wiki. Une los resultados (40 candidatos por consulta), deja como máximo un fragmento por archivo y se queda con los 5 mejores. Excluye los documentos que el usuario desactivó.
`evaluate_context`	Aplica el umbral de similitud y pide al LLM que verifique si los fragmentos restantes responden la pregunta.
`generate`	Responde solo con los fragmentos, en el idioma de la pregunta, citando el archivo de cada dato. Si el modelo concluye que no alcanza, se trata como rechazo.
`reject`	Devuelve el mensaje de rechazo explícito y sugiere subir documentos.
`save_history`	Agrega la pregunta y la respuesta a la memoria de la sesión y las guarda en la tabla `messages`.
`handle_error`	Si Ollama o Postgres fallan, avisa sin responder nada del lore; no se guarda en el historial y la API responde 503.

Estado y memoria

El estado (`AgentState`, un `TypedDict`) lleva la pregunta, el usuario, el idioma, la consulta reescrita, los fragmentos con su puntaje, la decisión de suficiencia, la respuesta, las fuentes, el indicador de rechazo, el error y los mensajes. El grafo se compila con un checkpointer (`InMemorySaver`) y cada conversación es un

`thread_id` con el usuario delante (`usuario:session_id`), así dos usuarios nunca comparten estado. Si el servidor se reinicia y se pierde esa memoria, el servicio de chat recupera los mensajes previos de la conversación desde la tabla `messages` y se los pasa al grafo.

8. Modelo local y base de datos vectorial

Modelo de lenguaje: qwen2.5:7b con Ollama

- **Por qué:** buen seguimiento de instrucciones para su tamaño, multilingüe (el usuario pregunta en español sobre una wiki en inglés) y, cuantizado, ocupa 4,7 GB: cabe en una GPU de consumo y en 8 GB de RAM.
- **Cómo se ejecuta:** Ollama en un contenedor; el backend lo usa con `ChatOllama` a temperatura 0,1, porque en RAG interesa la fidelidad al contexto y no la creatividad. Ollama mantiene el modelo cargado 24 horas para no recargarlo en cada pregunta.
- **Modelos de apoyo:** `bge-m3` para embeddings (1024 dimensiones, multilingüe), `qwen2.5vl:3b` para leer o describir imágenes y faster-whisper (`small`) para transcribir audio y video.
- **Limitaciones:** en CPU (la EC2 desplegada no tiene GPU) cada respuesta tarda del orden de minutos; con GPU son segundos. El modelo de 3b sería más rápido, pero en las pruebas el nodo de evaluación rechazaba con él todas las preguntas. Un modelo de 7b también comete errores al resumir textos largos, por eso las estrategias de la sección 10 no dependen solo de su criterio.

Base vectorial: PostgreSQL + pgvector

- `langchain_postgres.PGVector` guarda cada fragmento con su embedding y metadatos JSONB: `fuente` (archivo de origen), `title`, `juego`, `juegos`, `tipo`, `seccion`, `url` y `huella`.
- La búsqueda devuelve pares (fragmento, puntaje de relevancia entre 0 y 1) que se comparan con el umbral.
- Cada fragmento queda asociado a su documento por `fuente`: así se cita, se desactiva y se borra por archivo.
- La base inicial está versionada como respaldo (`backend/data/embeddings.dump`). Al arrancar con la base vacía, el servicio `db-seed` de Docker Compose la restaura, así el servidor no tiene que repetir la ingesta ni transcribir el video.

9. Proceso de carga de documentos

El mismo pipeline sirve para la base inicial (línea de comandos, `services/ingest_service.py`) y para los archivos que sube el usuario desde la web (`POST /documents`):

1. **Recibir:** el frontend envía el archivo como `multipart/form-data`. El backend limpia el nombre, valida la extensión y el tamaño (máximo `MAX_UPLOAD_MB`, 100 MB por defecto) y lo escribe en una carpeta temporal.
2. **Extraer:** según el formato.

Formato	Cómo se lee	Sección citable
.md (wiki)	Texto + frontmatter (title, juego, URL)	Título `##`
.txt	Texto plano	—
.pdf	Una página = un documento (pypdf)	Página N
.docx	Texto completo (docx2txt)	—
.csv	Una fila = un documento, "columna: valor"	Fila N
.jpg, .png	`qwen2.5vl:3b` transcribe el texto o, si no tiene, describe la imagen; si no es del juego, no se indexa	Títulos de la imagen
.mp3, .mp4	faster-whisper con los nombres del juego como *hotwords*, en bloques de 90 s	mm:ss – mm:ss

3. **Fragmentar:** `RecursiveCharacterTextSplitter` en trozos de 1000 caracteres con 150 de solapamiento, cortando primero por párrafos y frases. Cada fragmento empieza con la cabecera `Título (juego) > Sección`, para que el embedding sepa de qué trata aunque el título haya quedado en otro fragmento. Si el archivo no dice a qué juego pertenece, se deduce por fragmento según los nombres propios que menciona (Lothric → DS3, Vendrick → DS2).
4. **Embeber:** `bge-m3` convierte cada fragmento en un vector.
5. **Almacenar:** se guardan en pgvector con `fuente = uploads/<uuid>/<archivo>` y se registra el archivo en la tabla `documents` a nombre del usuario.
6. **Confirmar:** la API responde 201 con el nombre, el tipo, los juegos detectados y el número de fragmentos. Desde ese momento el agente puede usarlo.

Al borrar un documento se eliminan primero sus fragmentos y después la fila: si algo falla a mitad, el documento sigue visible y se puede reintentar, en vez de quedar fragmentos que el agente usa sin que nadie los vea.

10. Estrategias contra alucinaciones

El enunciado pide al menos dos; el agente implementa seis, en distintos puntos del flujo:

#	Estrategia	Implementación

1	Búsqueda obligatoria	El nodo `retrieve` se ejecuta siempre: ningún camino del grafo responde sin consultar la base. El clasificador de intención no puede rechazar por su cuenta.
2	Umbral mínimo de similitud	Se descartan los fragmentos con puntaje menor a `SIMILARITY_THRESHOLD = 0.46`. Entre 0,38 y 0,46 solo entran los que mencionan un nombre poco frecuente de la pregunta (por ejemplo "Gael"), porque un nombre suelto da puntajes bajos. Si no queda ninguno, se rechaza sin llamar al LLM.
3	Nodo de validación del contexto	`evaluate_context` pide al LLM que responda YES/NO a si los fragmentos responden la pregunta. Con NO, el grafo va a `reject`.
4	Prompt restrictivo	El prompt de respuesta obliga a usar solo los fragmentos, prohíbe el conocimiento previo, pide señalar contradicciones y aclara que la conversación sirve para entender la pregunta, no como fuente de datos.
5	Rechazo explícito	"No tengo suficiente información en mi base de conocimiento para responder esa pregunta" (o su versión en inglés), con sugerencia de subir documentos. Si el modelo llega a esa conclusión dentro de `generate`, la respuesta se marca como rechazo y no muestra fuentes.
6	Fuentes visibles	Cada dato cita entre paréntesis el archivo del que salió, y la respuesta lista sus fuentes. El frontend las muestra bajo cada respuesta y enlaza a la wiki cuando hay URL.

> Además, un fallo de Ollama o Postgres nunca produce una respuesta: el grafo lo desvía a `handle_error` y el usuario ve un aviso de servicio no disponible.

11. Seguridad y autenticación

- **Keycloak** gestiona registro, inicio y cierre de sesión. El realm `john-dark-soul` se importa desde un archivo versionado (`keycloak/realm-john-dark-soul.json`) y tiene un cliente público `frontend` con PKCE S256; los tokens de acceso duran 15 minutos y el frontend los renueva.
- **Validación en el backend** (`app/security.py`): lee el token del header `Authorization: Bearer`, descarga y cachea las llaves públicas del realm (JWKS) y verifica firma RS256, expiración, emisor (`iss`) y que el token fue emitido para el cliente `frontend` (`azp`). Si falla responde 401; si Keycloak no responde, 503.
- **Ningún endpoint queda expuesto:** los 11 dependen de `get_current_user`, y una prueba automática lo comprueba en cada ejecución.
- **Aislamiento por usuario:** historial, conversaciones, documentos y preferencias se filtran por el `sub` del token. Un documento de otro usuario responde 404, sin revelar que existe; las conversaciones del agente usan `usuario:session_id` como identificador.
- **CORS** solo acepta el origen del frontend (`FRONTEND_URL`).
- **Subidas:** nombre de archivo saneado (sin rutas), lista blanca de extensiones, límite de tamaño y procesamiento en una carpeta temporal que se borra al terminar.
- **En producción:** HTTPS en todo; solo Caddy expone puertos (80 y 443); Postgres, Ollama, Keycloak y el backend solo son accesibles dentro de la red de Docker. Las contraseñas viven en el `.env` del servidor, que no se sube al repositorio.

12. Despliegue

Componente	Dónde y cómo
Frontend	**Netlify**: https://johndarksoul.netlify.app. Compila `frontend/` con `npm run build` según `netlify.toml`, que además redirige las rutas de la SPA a `index.html`. Se redespliega con cada push a `main`.
Backend, Keycloak, modelos y base	**AWS EC2** `m7i-flex.large` (2 vCPU, 8 GB, capa gratuita), Ubuntu 24.04, Elastic IP 13.58.191.152, con Docker Compose: `docker-compose.yml` + `docker-compose.prod.yml`.
API	https://api.13-58-191-152.sslip.io (Swagger en `/docs`)
Keycloak	https://auth.13-58-191-152.sslip.io
HTTPS y dominios	Caddy obtiene certificados de Let's Encrypt; sslip.io resuelve `api.13-58-191-152.sslip.io` a la IP, sin comprar dominio.

Variables de entorno

Servidor (.env de la EC2)	Para qué
`API_DOMAIN`, `AUTH_DOMAIN`	Dominios públicos de la API y de Keycloak; de `AUTH_DOMAIN` se deriva `KEYCLOAK_URL`.

`FRONTEND_URL`	URL de Netlify; de ella salen el CORS y las redirecciones permitidas en Keycloak.
`POSTGRES_USER/PASSWORD/DB`, `DATABASE_URL`, `PG_COLLECTION`	Base de datos y colección vectorial.
`LLM_MODEL=qwen2.5:7b`, `EMBED_MODEL=bge-m3`, `VISION_MODEL`, `WHISPER_MODEL`, `TEMPERATURE`	Modelos locales.
`SIMILARITY_THRESHOLD`, `SIMILARITY_FLOOR`, `RETRIEVER_K`, `RETRIEVER_FETCH_K`, `CHUNK_SIZE`, `CHUNK_OVERLAP`, `MAX_UPLOAD_MB`	Recuperación, fragmentación y subidas.
`KEYCLOAK_ADMIN`, `KEYCLOAK_ADMIN_PASSWORD`, `KEYCLOAK_REALM`, `KEYCLOAK_CLIENT_ID`	Administración y realm de Keycloak.
Netlify	
`VITE_API_URL`, `VITE_KEYCLOAK_URL`, `VITE_KEYCLOAK_REALM`, `VITE_KEYCLOAK_CLIENT_ID`, `VITE_USE_MOCKS`	Se fijan al compilar: cada cambio requiere volver a desplegar.

Proceso

1. Netlify: importar el repositorio y reservar la URL del sitio.
2. AWS: lanzar la instancia (Ubuntu 24.04, 30–50 GB gp3, puertos 22, 80 y 443) y asociarle una Elastic IP.
3. En la instancia: `apt upgrade`, instalar Docker, clonar el repositorio, crear el `.env` desde `deploy/.env.production.example` y ejecutar `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build`. La base vectorial se restaura sola.
4. Descargar los modelos con `ollama pull` y ejecutar `deploy/configurar_keycloak.sh`, que autoriza la URL de Netlify en el cliente del realm.
5. Configurar las variables `VITE_*` en Netlify y volver a desplegar.

El paso a paso completo y la tabla de problemas frecuentes están en `docs/DEPLOYMENT.md`.

13. Evidencias de funcionamiento

Capturas tomadas sobre la aplicación desplegada (carpeta `docs/screenshots/`).

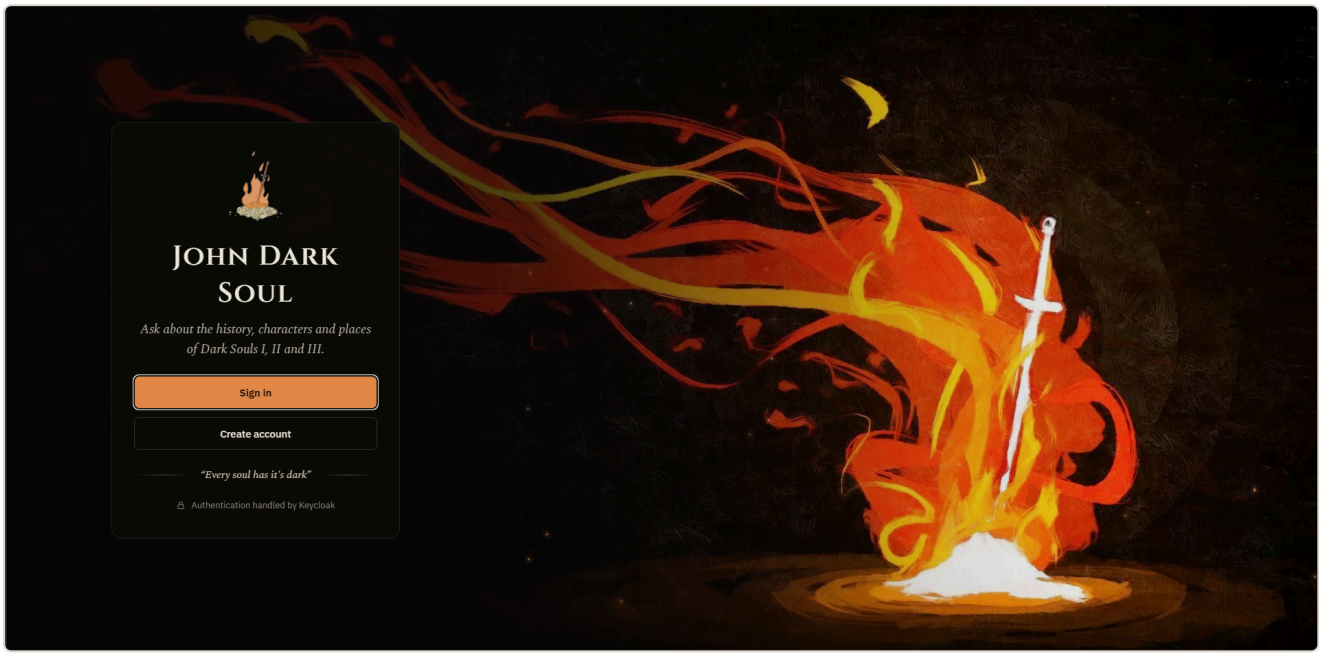


Figura 3. Pantalla de inicio de sesión; la autenticación la gestiona Keycloak.

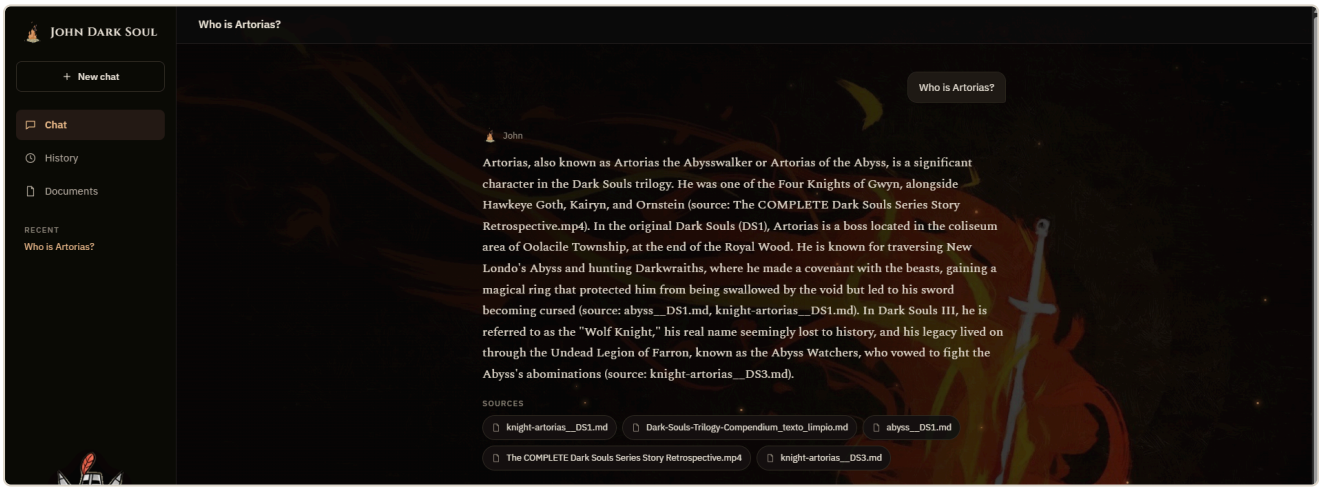


Figura 4. Pregunta respondida con citas por dato y la lista de fuentes usadas.

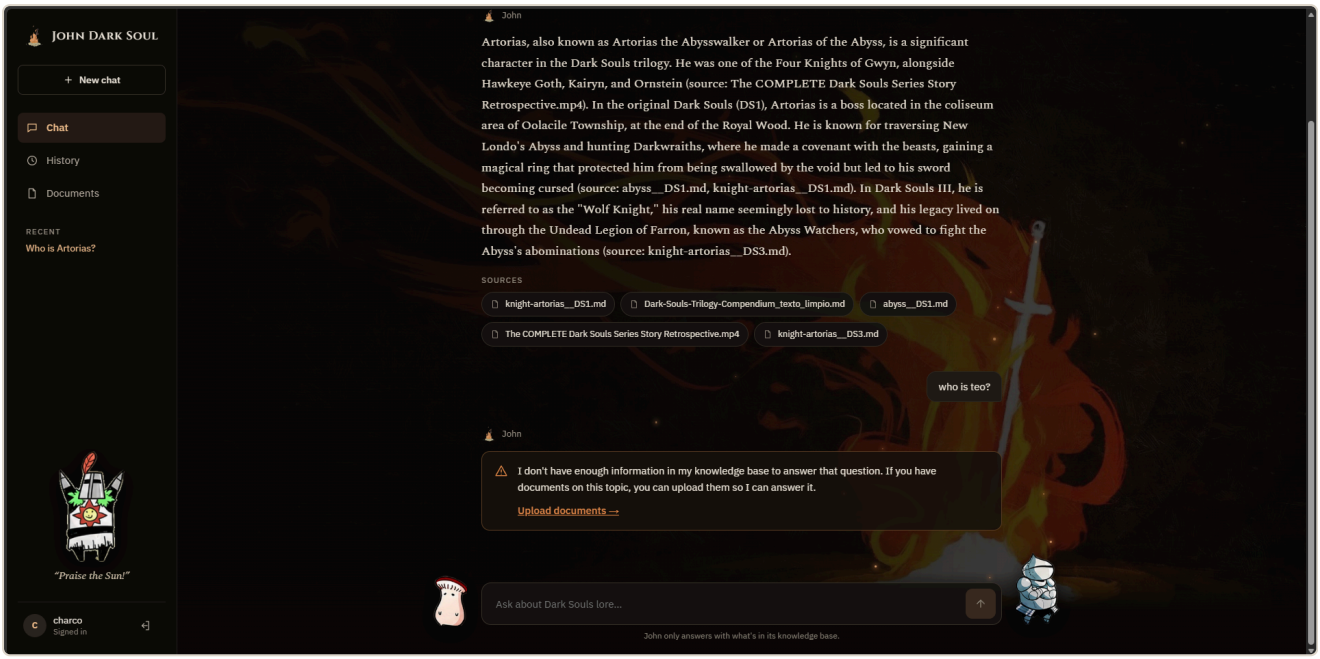


Figura 5. Pregunta sin información en la base: rechazo explícito y enlace para subir documentos.

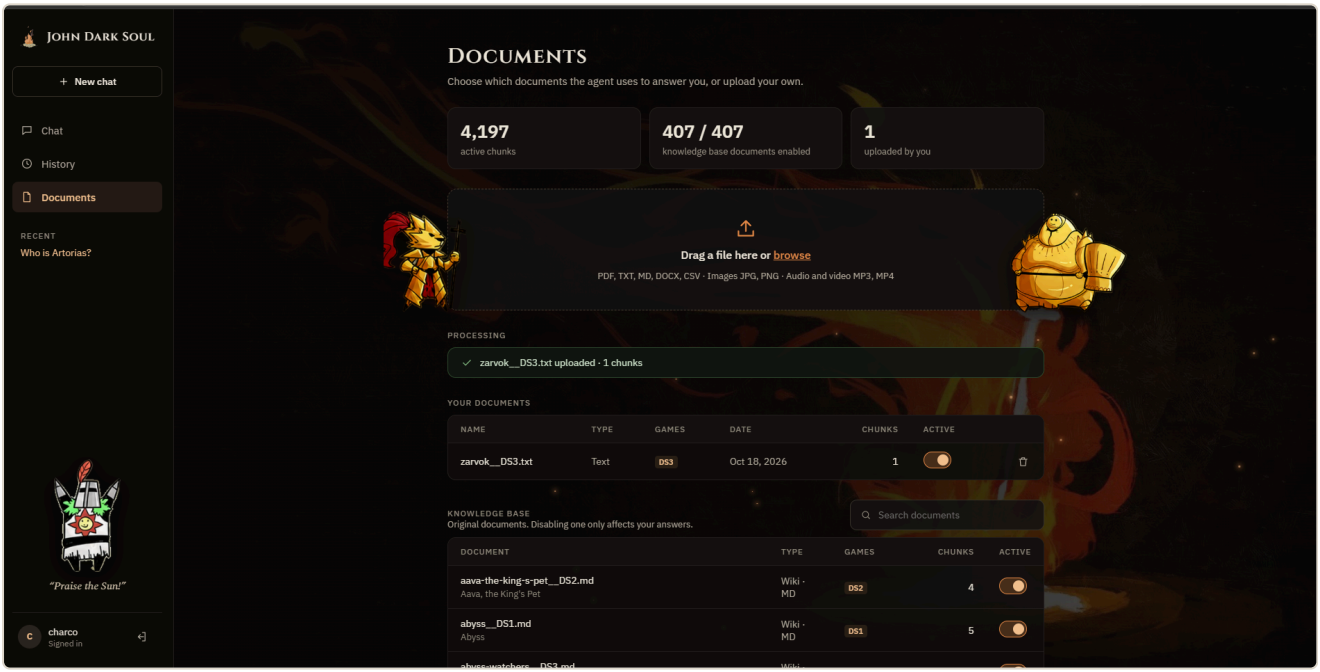


Figura 6. Documento subido y procesado (1 fragmento, juego DS3 detectado), con la base de conocimiento original debajo.

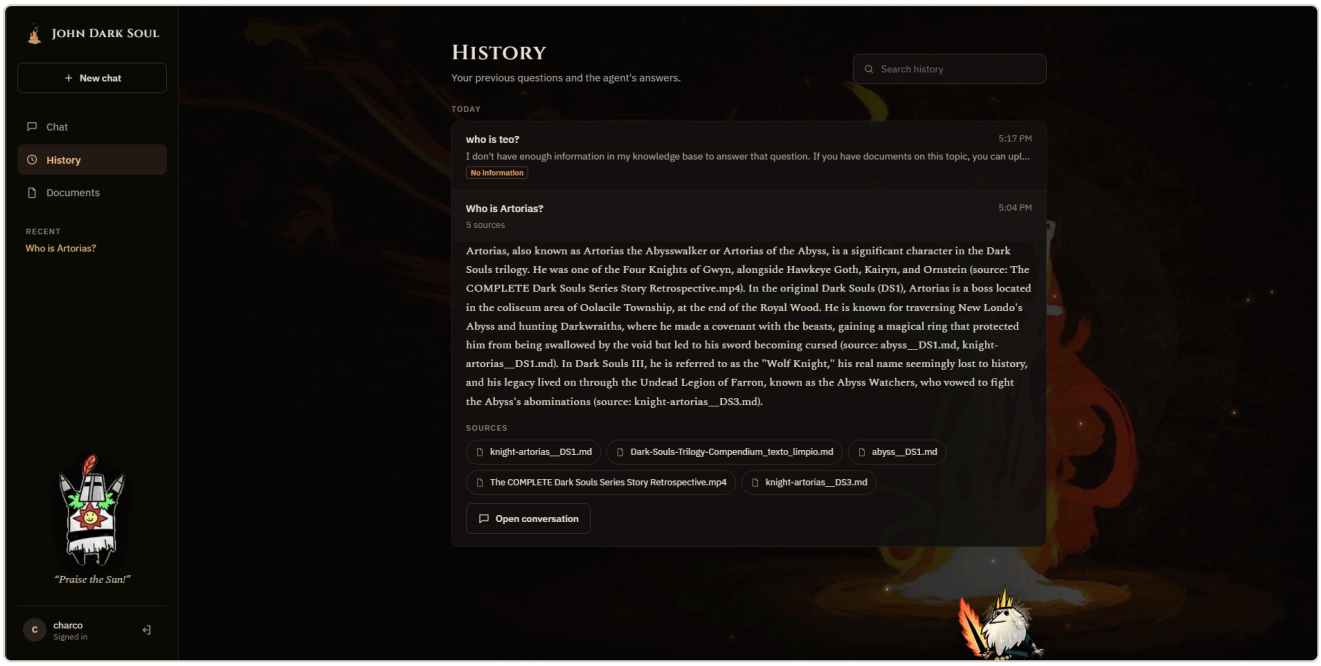


Figura 7. Historial: preguntas por día, respuesta completa con fuentes y acceso a la conversación.

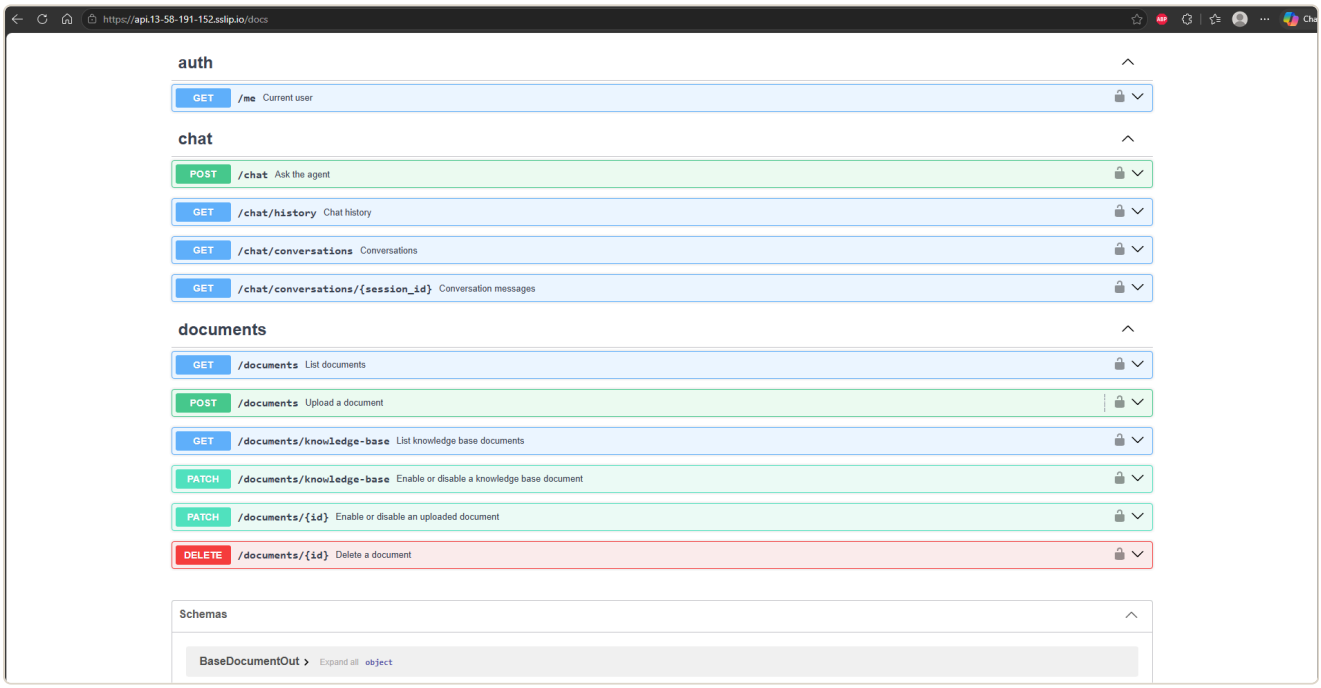


Figura 8. Los 11 endpoints de la API desplegada; el candado indica que todos exigen token.

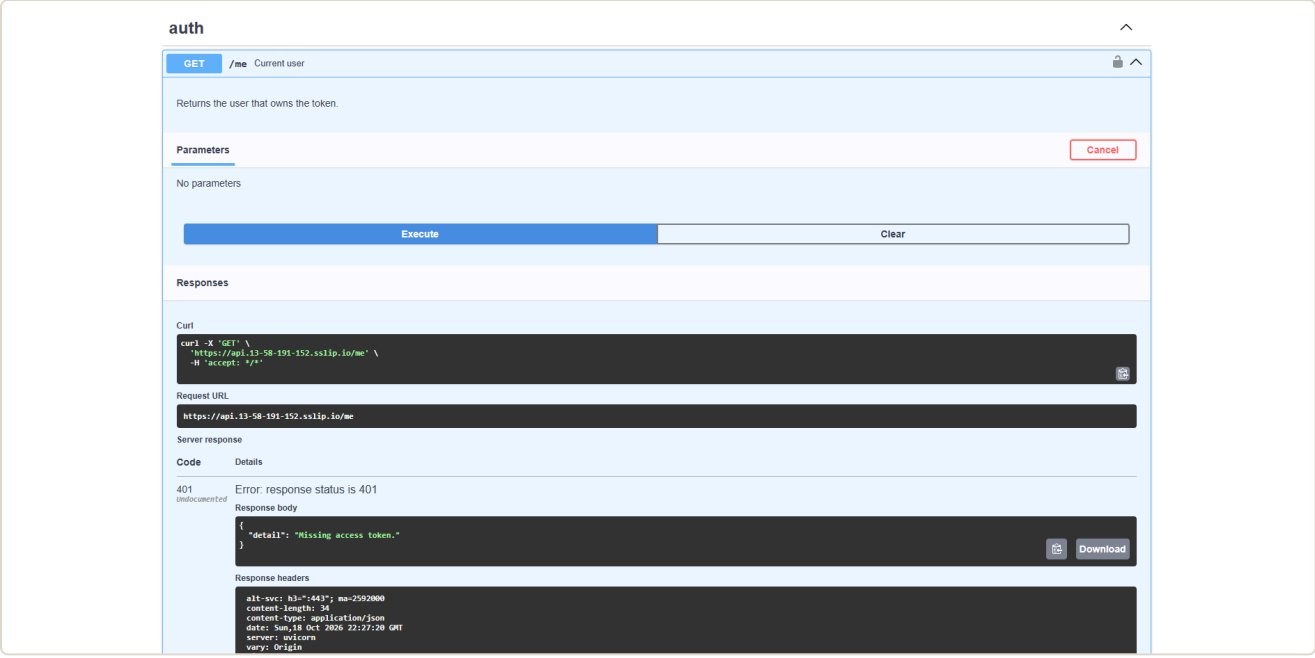


Figura 9. Sin token, la API responde 401 "Missing access token."

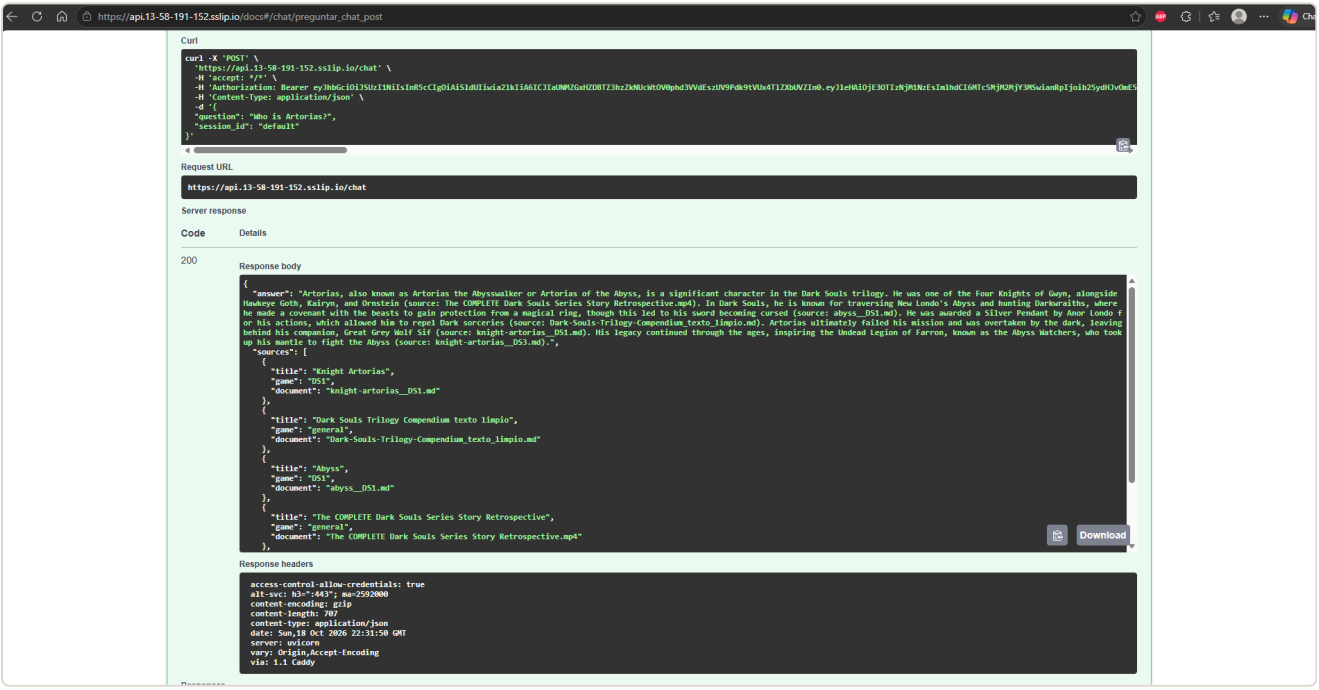


Figura 10. `POST /chat` sobre HTTPS en la EC2 (cabecera `via: 1.1 Caddy`): respuesta con sus fuentes.

14. Problemas encontrados y conclusiones

Problemas y cómo se resolvieron

Problema	Solución
El clasificador de intención rechazaba preguntas válidas sobre nombres que no conocía (Gael, Midir) o sobre documentos subidos por el usuario.	El clasificador pasó a decidir **cómo** buscar, no **si** buscar: la búsqueda es obligatoria y solo el umbral y `evaluate_context` pueden rechazar.
Un umbral fijo dejaba fuera preguntas con un nombre suelto ("who is gael"), cuyo puntaje es bajo aunque el fragmento sea el correcto.	Calibración con preguntas reales y un piso (0,38): entre el piso y el umbral entran solo los fragmentos que mencionan el nombre preguntado.
Preguntas en español sobre una wiki en inglés recuperaban mal.	Embeddings multilingües (`bge-m3`) y reescritura de la pregunta al inglés con nombres oficiales, buscando con ambas versiones.
Una página o el video llenaban todo el contexto y tapaban la página que respondía.	Máximo un fragmento por archivo en el contexto final.
Whisper en GPU fallaba en Windows por las librerías de CUDA.	Carga explícita de las DLL de NVIDIA y, si la GPU falla, repetición automática en CPU.
El navegador bloquea un backend HTTP desde Netlify (HTTPS) y `keycloak-js` exige HTTPS para PKCE.	Caddy como proxy con certificados automáticos de Let's Encrypt y dominios de sslip.io, sin comprar dominio.
Keycloak en modo producción no creaba su esquema en Postgres.	Tarea de un solo uso (`keycloak-schema`) en el compose de producción que lo crea antes de arrancarlo.
Con la base recién creada, la primera carga de la app daba 503: peticiones simultáneas creaban las mismas tablas a la vez.	*Advisory lock* de Postgres en `init_db`.
El plan gratuito de AWS no permite instancias de 16 GB.	Despliegue en `m7i-flex.large` (8 GB), la mayor del plan gratuito.
`qwen2.5:3b` era más rápido en CPU, pero el nodo de evaluación rechazaba con él todas las preguntas.	Se mantuvo `qwen2.5:7b`, aceptando respuestas más lentas en CPU.
Let's Encrypt no podía emitir los certificados ("Timeout during connect").	La IP estaba mal escrita en el `.env` (sslip.io resuelve exactamente lo escrito). Al corregirla, Caddy obtuvo los certificados.

Conclusiones

- **Un grafo explícito da control.** Con LangGraph el flujo es código revisable: el modelo no puede saltarse la búsqueda ni responder sin pasar por la evaluación del contexto, algo que un agente que elige sus herramientas no garantiza.
- **Las alucinaciones se controlan por capas.** Ninguna estrategia basta sola: el umbral filtra por similitud, el nodo de evaluación juzga el contenido, el prompt limita la generación y las fuentes permiten al usuario comprobarla.
- **Calibrar con preguntas reales fue necesario** para equilibrar respuestas y rechazos: un umbral demasiado alto rechaza preguntas válidas y uno bajo deja pasar contexto irrelevante.
- **Un modelo local es viable, con costo en velocidad.** Con GPU responde en segundos; en la nube sin GPU, en minutos. La calidad del modelo pesa: el de 3b no sostuvo el control de alucinaciones.
- **Desplegar pronto revela problemas** que no aparecen en local (HTTPS obligatorio, esquemas, carreras al crear tablas, límites de la capa gratuita). Simular la producción antes de crear la instancia ahorró tiempo y costo.